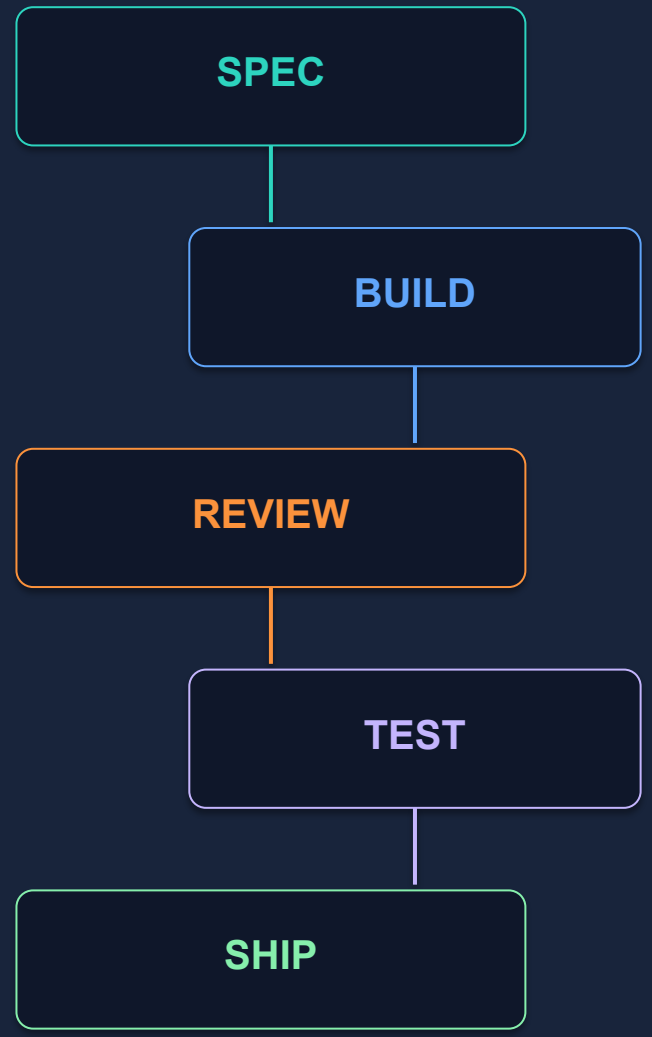
FROM AI CODING TO AUTONOMOUS SOFTWARE ENGINEERING

Construindo uma software factory orientada por agentes

Curso intensivo • 4 horas • Conceitos + demonstração prática

Agentes • Harness Engineering • Skills • Spec-Driven Development • QA



A promessa — e o problema

A meta não é gerar mais código. É entregar mudanças confiáveis com menos coordenação manual.

MAIS VELOCIDADE

Agentes investigam, implementam e verificam tarefas usando ferramentas reais.

MAIS CONTROLE

Políticas, checkpoints e permissões delimitam o que pode ser feito.

MAIS CONFIANÇA

Testes, evidências e observabilidade verificam o resultado — não apenas a resposta.

Pergunta-guia: o que precisa ser verdade para delegarmos uma tarefa de software a um agente?

Ao final, você será capaz de…

Foco avançado: projetar sistemas agentic, não apenas operar um assistente de código.

- **Distinguir**
  LLM, workflow determinístico, agente e sistema multiagente.
- **Projetar**
  Um ciclo de software com especificação, implementação, revisão e QA.
- **Controlar**
  Ferramentas, estado, memória, retries, orçamento e autonomia.
- **Avaliar**
  Qualidade, risco, custo, observabilidade e critérios de escalonamento.

Quatro horas, uma única história

A mesma funcionalidade evolui do requisito até a decisão de merge.

00:00–00:45	**Fundamentos de agentes**	45 min
00:45–01:45	**Spec-Driven Development + implementação**	60 min
01:45–02:00	**Intervalo**	15 min
02:00–03:00	**Agentes no ciclo de software**	60 min
03:00–04:00	**Autonomia, risco e produção**	60 min

Do prompt ao sistema agentic

A autonomia não é uma propriedade binária: ela emerge do loop, das ferramentas e dos limites.

PROMPT

Entrada → resposta

O modelo não executa ações externas por conta própria.

WORKFLOW

Etapas predefinidas

A lógica e a ordem são majoritariamente codificadas.

AGENTE

Decide o próximo passo

Usa ferramentas, observa resultados e adapta o plano.

SISTEMA AGENTIC

Vários componentes

Estado, políticas, eventos e supervisão coordenam o trabalho.

Regra prática: use autonomia onde a incerteza exige decisão; use código determinístico onde a garantia é obrigatória.

O loop de um agente

Um agente útil fecha o ciclo entre intenção, ação e evidência.



Saída inválida? Corrigir, tentar novamente com limite ou escalar. Não repetir indefinidamente.

As peças de um agente de engenharia

O modelo é só uma parte do sistema.

Modelo

Raciocínio, síntese e seleção de ações

Contexto

Requisitos, código, convenções e histórico

Tools / MCP

Acesso controlado a Git, CI, APIs, Linear, Sentry

Estado e memória

Progresso, decisões, tentativas e resultados

Políticas

Permissões, limites, critérios de parada

Avaliação

Testes, evidências, métricas e logs

Autonomia precisa de fronteiras

Defina a autonomia por tipo de ação — não apenas por um interruptor 'human-in-the-loop'.

LEITURA	**Consultar código, issues, logs**	Pode ser ampla, com escopo definido
ALTERAÇÃO	**Editar arquivos, criar branch**	Sandbox, diff e testes
PUBLICAÇÃO	**Criar PR, comentar, atualizar card**	Permissões mínimas e trilha de auditoria
AÇÃO CRÍTICA	**Merge, deploy, alteração de dados**	Aprovação, políticas e rollback

Princípio: menor privilégio + evidência verificável + possibilidade de interrupção.

Uma tarefa real: cadastrar usuários

Vamos acompanhar uma única feature em todas as etapas.

REQUISITO

Criar endpoint de cadastro

- validação de e-mail
- senha com hash
- resposta consistente
- testes de sucesso e erro

RESTRIÇÕES

- manter arquitetura existente
- não expor dados sensíveis
- não quebrar contratos
- migrações reversíveis

DEFINIÇÃO DE PRONTO

- critérios de aceitação passam
- testes automatizados verdes
- revisão sem bloqueadores
- PR documentada

Contexto para a demonstração: repositório pequeno, testes existentes e uma falha reproduzível.

Antes do código: transformar intenção em contrato

A especificação reduz ambiguidade e torna o trabalho verificável.

PRÉ-CONDIÇÕES

O que deve ser verdade antes?

- autenticação/escopo definidos
- dependências disponíveis
- contrato de API conhecido

PÓS-CONDIÇÕES

O que deve ser verdade depois?

- usuário persistido corretamente
- senha armazenada como hash
- resposta sem campos sensíveis

CRITÉRIOS DE ACEITAÇÃO

Como comprovar?

- status e payload esperados
- entradas inválidas rejeitadas
- duplicidade tratada
- testes reproduzíveis

Uma boa spec é pequena o suficiente para orientar a implementação e precisa o suficiente para ser testada.

O pipeline de implementação

O agente pode planejar de forma flexível; os gates de qualidade precisam ser explícitos.



Em cada etapa: registrar evidência, atualizar estado e definir o próximo passo.

Harness Engineering: o ambiente molda o comportamento

Não dependa apenas de instruções no prompt para garantir consistência.

CONTEXTO E CONVENÇÕES

CLAUDE.md / instruções do projeto

Arquitetura, comandos, padrões, limites e caminhos relevantes.

CHECKPOINTS

Skills e scripts verificáveis

Revisão de arquitetura, testes obrigatórios, formato de entrega.

EXECUÇÃO CONTROLADA

Sandbox, permissões e CI

Ações limitadas, logs, orçamento e condição de parada.

Harness = infraestrutura + políticas + contexto + verificações que tornam o agente operacional.

Skills: instruções reutilizáveis, gates executáveis

Separar orientação do modelo de verificações que o sistema consegue impor.

SKILL DE IMPLEMENTAÇÃO

- padrões do repositório
- passos de investigação
- convenções arquiteturais
- formato de saída

SKILL DE CODE REVIEW

- segurança e autorização
- schema e migrações
- complexidade e escalabilidade
- riscos e evidências

GATES DETERMINÍSTICOS

- testes obrigatórios
- lint / typecheck
- secret scanning
- status checks bloqueantes

A Skill pode orientar. O gate deve bloquear quando a regra é obrigatória.

PAUSA

15 minutos

Na volta: conectar agentes especializados ao ciclo completo de software.

Uma software factory orientada por agentes

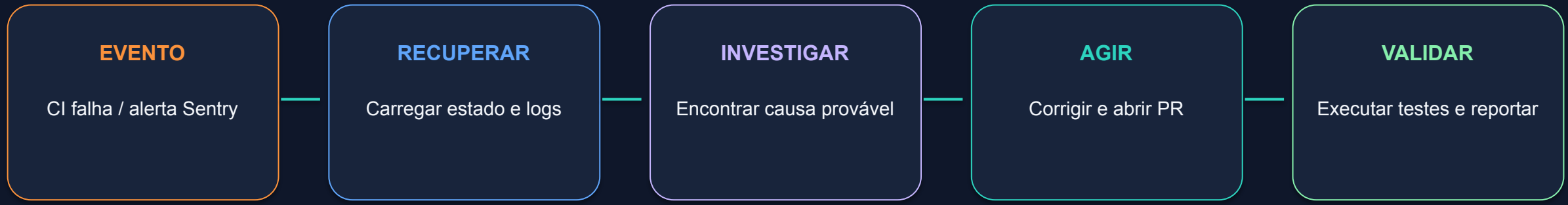
Papéis especializados compartilham um contrato de trabalho e evidências.



Contrato compartilhado: objetivo • estado • artefatos • evidências • limites • condição de parada

Agentes orientados a eventos e tarefas long-running

O processo pode sobreviver a esperas, falhas, reinícios e eventos externos.



PERSISTÊNCIA

Estado, checkpoints, histórico

RESILIÊNCIA

Retries limitados, idempotência

OBSERVABILIDADE

Logs, traces, custo, resultado

Code review: agente não é autoridade final

O valor vem de encontrar problemas com evidências reproduzíveis.

ARQUITETURA

Acoplamento, boundaries, responsabilidade

DADOS

Schema, índices, migrações, concorrência

SEGURANÇA

Autorização, segredos, validação, exposição

COMPLEXIDADE

Big-O, N+1, escalabilidade, latência

Formato de achado recomendado

Severidade • arquivo/linha • evidência • impacto • recomendação • teste que comprova

Evite comentários vagos, duplicados ou sem caminho de reprodução.

QA agent: validar o comportamento, não a intenção

A especificação vira um conjunto de verificações executáveis.

UNITÁRIOS

Regras e casos de borda

INTEGRAÇÃO

API, banco e dependências

E2E / BROWSER

Fluxo do usuário no sistema

REGRESSÃO

Comportamentos já existentes

Loop de correção Falha → evidência → hipótese → correção → reexecução → relatório

Memória e estado não são a mesma coisa

Separar fatos duráveis do progresso operacional de uma execução.

ESTADO DA EXECUÇÃO

- tarefa atual e etapa
- tentativas e checkpoints
- ferramentas já executadas
- resultados e erros
- próxima ação permitida

MEMÓRIA / CONHECIMENTO

- convenções do projeto
- decisões arquiteturais
- soluções recorrentes
- padrões de falha
- links para fontes de verdade

Persistir o que permite retomar com segurança; evitar acumular contexto irrelevante ou desatualizado.

Confiabilidade: falhas fazem parte do desenho

Agentes operam em ambientes incertos. A arquitetura precisa prever isso.

Timeouts

Toda chamada externa tem limite

Retries

Backoff e máximo de tentativas

Idempotência

Repetir não deve duplicar efeitos

Checkpoints

Salvar progresso recuperável

Cancelamento

Interromper execução e revogar ação

Observabilidade

Correlacionar evento, ação e resultado

Condição de parada explícita > loop infinito de 'tentar até funcionar'.

Score de risco para PRs: priorizar supervisão

O score apoia a decisão; não substitui gates nem regras de segurança.

SINAIS DE ENTRADA

- Arquivos e linhas alterados
- Autenticação, autorização e migrações
- Resultados de testes e análise estática
- Histórico de falhas semelhantes

MODELO

P(falha)

Calibrar com histórico

POLÍTICA DE DECISÃO

BAIXO RISCO

Checks verdes + política aprovada

MÉDIO RISCO

Revisão reforçada

ALTO RISCO

Bloquear / exigir humano

Nunca autorizar merge só porque o LLM declarou alta confiança.

Métricas: medir o sistema, não a impressão

Sem avaliação, 'autonomia' pode apenas esconder trabalho de correção.

QUALIDADE

Taxa de testes aprovados
Defeitos escapados
Reabertura de PR

EFICIÊNCIA

Lead time
Tempo humano poupado
Retries por tarefa

CUSTO

Tokens e custo por tarefa
Tempo de execução
Uso de ferramentas

SEGURANÇA

Ações bloqueadas
Escalonamentos
Incidentes / regressões

Compare com uma linha de base humana e acompanhe tendências por tipo de tarefa.

Stack mínima para a demonstração

Escolha ferramentas conhecidas e invista o tempo em contratos, controles e evidências.

Interface / tarefa	Linear ou issue do GitHub
Agente	Claude Code ou Cursor
Ferramentas	GitHub, terminal, MCPs autorizados
Gates	CI, testes, lint, typecheck, scanners
Observabilidade	Logs, métricas, Sentry quando aplicável
Persistência	PostgreSQL ou storage para checkpoints

Não é obrigatório adotar todos os componentes para demonstrar o padrão.

Desenhe sua software factory

Trabalho em pequenos grupos • 10–15 minutos

1. ESCOLHA UMA TAREFA

Ex.: corrigir bug, criar endpoint, atualizar dependência ou responder a alerta.

2. DEFINA OS AGENTES

Quem especifica, implementa, revisa e testa? Quais ferramentas cada um pode usar?

3. DEFINA OS LIMITES

Quais gates são obrigatórios? Quando parar? Quando exigir aprovação humana?

Entregável: um diagrama de fluxo + critérios de sucesso + política de escalonamento.

Checklist antes de colocar agentes em produção

Se não consegue responder a estas perguntas, reduza a autonomia.

- Qual é o objetivo e como o sucesso é medido?
- Quais dados e ferramentas o agente pode acessar?
- Quais ações exigem confirmação ou são proibidas?
- Como detectar erro, repetição e saída inválida?
- Como retomar, cancelar e auditar uma execução?
- Quais gates determinísticos bloqueiam a entrega?
- Qual é o custo máximo e a condição de parada?

A IDEIA CENTRAL

Autonomia não é ausência de controle.

É a capacidade de agir dentro de limites explícitos,
produzindo evidências verificáveis e escalando quando necessário.

OBRIGADO